Robot Motion Planning and Manipulator Dynamics

Om Patel (23BEC211D)

Common setup

- Language and libraries: Python 3 with numpy and matplotlib. scipy.integrate.solve_ivp (RK45, the Python equivalent of ode45) is used only for the ODEs in Task 4.
- Dijkstra, A* and RRT are implemented from scratch (own priority queue, own collision checks). No built-in planner or shortest-path function is used.
- Parameters that were not given are assumed and stated at the start of each task.
- All the codes are available at - https://github.com/Flux-15/Robot-Motion-Planning-and-Manipulator-Dynamics.git

Task 1: Grid-based path planning: Dijkstra and A*

Problem and assumptions

30 x 30 grid, start (2, 2), goal (27, 27), obstacles R1 (x 8-10, y 0-18), R2 (x 15-17, y 10-29), R3 (x 22-24, y 0-20), limits inclusive. 8-connected moves with cost 1 (straight) and sqrt(2) (diagonal); a diagonal move needs both orthogonal neighbours free (no corner cutting). A* heuristic: Euclidean distance to the goal.

- Assumed: ties in the priority queue are broken by insertion order; a node counts as expanded when it is popped and closed (the goal included).
- Assumed: computation time is the mean of 20 runs measured with time.perf_counter (machine dependent; use the ratio, not the absolute value).
- Assumed: line-of-sight is tested by sampling the segment every 0.01 cell and rejecting it if it touches an obstacle cell (cell (i, j) covers i +/- 0.5, j +/- 0.5). Heading changes are wrapped to [-pi, pi]; a turn is any heading change above 1e-9 rad.

Code

```
import heapq
import json
import math
import time
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

N = 30
START, GOAL = (2, 2), (27, 27)
OBSTACLES = [  # (x_min, x_max, y_min, y_max), inclusive
    (8, 10, 0, 18),   # R1
    (15, 17, 10, 29), # R2
    (22, 24, 0, 20),  # R3
]
SQRT2 = math.sqrt(2.0)

grid = np.zeros((N, N), dtype=bool)  # grid[y, x] = True -> obstacle
for x0, x1, y0, y1 in OBSTACLES:
    grid[y0:y1 + 1, x0:x1 + 1] = True

def free(x, y):
    return 0 <= x < N and 0 <= y < N and not grid[y, x]

MOVES = [(1, 0, 1.0), (-1, 0, 1.0), (0, 1, 1.0), (0, -1, 1.0),
         (1, 1, SQRT2), (1, -1, SQRT2), (-1, 1, SQRT2), (-1, -1, SQRT2)]

def neighbours(node):
    x, y = node
    for dx, dy, c in MOVES:
        nx, ny = x + dx, y + dy
        if not free(nx, ny):
            continue
        # no corner cutting: both orthogonal neighbours must be free
        if dx != 0 and dy != 0 and not (free(x + dx, y) and free(x, y + dy)):
            continue
        yield (nx, ny), c

def euclid(a, b):
    return math.hypot(a[0] - b[0], a[1] - b[1])

def search(start, goal, use_heuristic):
    """Dijkstra (use_heuristic=False) or A* (True).
    Returns path, cost, expanded-node list (in expansion order)."""
    h = (lambda n: euclid(n, goal)) if use_heuristic else (lambda n: 0.0)
    g = {start: 0.0}
    parent = {start: None}
```

```
    closed = set()
    expanded = []
    counter = 0
    pq = [(h(start), 0, start)]
    while pq:
        _, _, node = heapq.heappop(pq)
        if node in closed:
            continue
        closed.add(node)
        expanded.append(node)
        if node == goal:
            break
        for nb, c in neighbours(node):
            ng = g[node] + c
            if nb not in g or ng < g[nb] - 1e-12:
                g[nb] = ng
                parent[nb] = node
                counter += 1
                heapq.heappush(pq, (ng + h(nb), counter, nb))
    if goal not in parent:
        return None, math.inf, expanded
    path, n = [], goal
    while n is not None:
        path.append(n)
        n = parent[n]
    return path[::-1], g[goal], expanded

def dijkstra(start, goal):
    return search(start, goal, False)

def astar(start, goal):
    return search(start, goal, True)

def path_length(path):
    return sum(euclid(path[i], path[i + 1]) for i in range(len(path) - 1))

def turn_metrics(path):
    """Returns (number of turns, total absolute heading change in rad)."""
    headings = [math.atan2(path[i + 1][1] - path[i][1], path[i + 1][0] - path[i][0])
                for i in range(len(path) - 1)]
    turns, total = 0, 0.0
    for a, b in zip(headings[:-1], headings[1:]):
        d = (b - a + math.pi) % (2 * math.pi) - math.pi  # wrap to [-pi, pi]
        if abs(d) > 1e-9:
            turns += 1
            total += abs(d)
    return turns, total

def mean_runtime(fn, runs=20):
    ts = []
    for _ in range(runs):
        t0 = time.perf_counter()
        fn(START, GOAL)
        ts.append(time.perf_counter() - t0)
    return 1000.0 * float(np.mean(ts))  # ms

def line_of_sight(a, b, step=0.01):
    """True if the straight segment a->b stays in free cells.
    Cell (i, j) occupies [i-0.5, i+0.5] x [j-0.5, j+0.5]. The segment is
    sampled densely; every sample's cell (and the cells touched within a
    small margin) must be free, which also blocks squeezing through
    diagonal gaps between obstacle corners."""
    (x0, y0), (x1, y1) = a, b
    L = math.hypot(x1 - x0, y1 - y0)
    n = max(2, int(L / step))
    eps = 1e-6
    for k in range(n + 1):
        t = k / n
        x, y = x0 + t * (x1 - x0), y0 + t * (y1 - y0)
        # round half away from the cell boundary conservatively: check both sides
        for cx in {math.floor(x + 0.5 - eps), math.floor(x + 0.5 + eps)}:
            for cy in {math.floor(y + 0.5 - eps), math.floor(y + 0.5 + eps)}:
                if not free(cx, cy):
                    return False
    return True

def smooth_path(path):
    """Greedy line-of-sight shortcutting: from the current anchor, jump to the
    farthest path vertex that is directly visible."""
    out = [path[0]]
    i = 0
    while i < len(path) - 1:
        j = len(path) - 1
        while j > i + 1 and not line_of_sight(path[i], path[j]):
            j -= 1
        out.append(path[j])
        i = j
    return out

def plot_result(title, path, expanded, fname, smooth=None):
    fig, ax = plt.subplots(figsize=(8, 8))
    cmap = ListedColormap(["white", "black"])
    ax.imshow(grid, origin="lower", cmap=cmap, extent=(-0.5, N - 0.5, -0.5, N - 0.5))
    if expanded:
        ex = np.array(expanded)
        ax.scatter(ex[:, 0], ex[:, 1], marker="s", s=38, c="lightskyblue",
                   edgecolors="none", label=f"Expanded ({len(expanded)})")
    if path:
        p = np.array(path)
        ax.plot(p[:, 0], p[:, 1], "r-", lw=2.2, label="Path")
    if smooth:
        s = np.array(smooth)
        ax.plot(s[:, 0], s[:, 1], "g-", lw=2.5, marker="o", ms=4, label="Smoothed path")
    ax.plot(*START, "go", ms=11, mec="k", label="Start (2,2)")
    ax.plot(*GOAL, "m*", ms=17, mec="k", label="Goal (27,27)")
    ax.set_xticks(range(0, N, 2))
    ax.set_yticks(range(0, N, 2))
```

```
    ax.set_xticks(np.arange(-0.5, N, 1), minor=True)
    ax.set_yticks(np.arange(-0.5, N, 1), minor=True)
    ax.grid(which="minor", color="lightgrey", lw=0.4)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_title(title)
    ax.legend(loc="upper left", fontsize=8, framealpha=0.95)
    fig.tight_layout()
    fig.savefig(fname, dpi=150)
    plt.close(fig)

def main():
    d_path, d_cost, d_exp = dijkstra(START, GOAL)
    a_path, a_cost, a_exp = astar(START, GOAL)
    assert d_path and a_path, "No path found"

    d_time = mean_runtime(dijkstra)
    a_time = mean_runtime(astar)
    d_turns, d_head = turn_metrics(d_path)
    a_turns, a_head = turn_metrics(a_path)

    # (b) plots
    plot_result("Dijkstra", d_path, d_exp, "dijkstra.png")
    plot_result("A* (Euclidean heuristic)", a_path, a_exp, "astar.png")

    # (c) comparison table
    print("\n(c) Comparison")
    print(f"{'Metric':<32}{'Dijkstra':>12}{'A*':>12}")
    print("-" * 56)
    print(f"{'Path cost':<32}{d_cost:>12.4f}{a_cost:>12.4f}")
    print(f"{'Nodes expanded':<32}{len(d_exp):>12d}{len(a_exp):>12d}")
    print(f"{'Mean time of 20 runs (ms)':<32}{d_time:>12.3f}{a_time:>12.3f}")
    print(f"{'Number of turns':<32}{d_turns:>12d}{a_turns:>12d}")
    print(f"{'Total heading change (rad)':<32}{d_head:>12.4f}{a_head:>12.4f}")

    # (d) smoothing
    s_path = smooth_path(a_path)
    s_len = path_length(s_path)
    s_turns, s_head = turn_metrics(s_path)
    plot_result("A* path with line-of-sight smoothing", a_path, a_exp,
                "astar_smoothed.png", smooth=s_path)

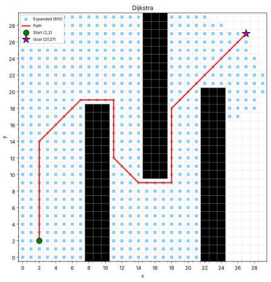
    print("\n(d) Smoothed A* path")
    print(f"Waypoints           : {s_path}")
    print(f"Length              : {s_len:.4f}  (raw A*: {path_length(a_path):.4f})")
    print(f"Number of turns     : {s_turns}")
    print(f"Total heading change : {s_head:.4f} rad")

    json.dump({
        "dijkstra": {"cost": d_cost, "expanded": len(d_exp), "time_ms": d_time,
                     "turns": d_turns, "heading": d_head},
        "astar": {"cost": a_cost, "expanded": len(a_exp), "time_ms": a_time,
                  "turns": a_turns, "heading": a_head},
        "smooth": {"length": s_len, "turns": s_turns, "heading": s_head,
                   "waypoints": s_path, "raw_length": path_length(a_path)},
    }, open("results_task1.json", "w"), indent=1)

    # side-by-side summary figure
    fig, axes = plt.subplots(1, 3, figsize=(18, 6.3))
    for ax, (t, p, e, s) in zip(axes, [
        (f"Dijkstra ({len(d_exp)} expanded)", d_path, d_exp, None),
        (f"A* ({len(a_exp)} expanded)", a_path, a_exp, None),
        ("A* + smoothing", a_path, [], s_path)]):
        ax.imshow(grid, origin="lower", cmap=ListedColormap(["white", "black"]),
                  extent=(-0.5, N - 0.5, -0.5, N - 0.5))
        if e:
            ex = np.array(e)
            ax.scatter(ex[:, 0], ex[:, 1], marker="s", s=14, c="lightskyblue", edgecolors="none")
        pp = np.array(p)
        ax.plot(pp[:, 0], pp[:, 1], "r-", lw=1.8)
        if s:
            ss = np.array(s)
            ax.plot(ss[:, 0], ss[:, 1], "g-", lw=2.5, marker="o", ms=4)
        ax.plot(*START, "go", ms=9, mec="k")
        ax.plot(*GOAL, "m*", ms=14, mec="k")
        ax.set_title(t)
        ax.set_aspect("equal")
    fig.tight_layout()
    fig.savefig("comparison.png", dpi=130)
    plt.close(fig)

if __name__ == "__main__":
    main()
```

Results (parts a, b)



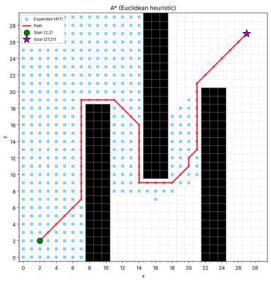


Figure 1: Dijkstra: obstacles (black), expanded cells (blue squares), final path (red), start and goal.

Figure 2: A (Euclidean heuristic): obstacles, expanded cells, final path, start and goal.*

Comparison (part c)

Table 1: Dijkstra vs A (time = mean of 20 runs).*

Metric	Dijkstra	A*
Path cost	60.0416	60.0416
Nodes expanded	610	417
Computation time [ms]	2.440	1.862
Number of turns	7	10
Total heading change [rad]	7.0686	9.4248

Smoothing of the A* path (part d)

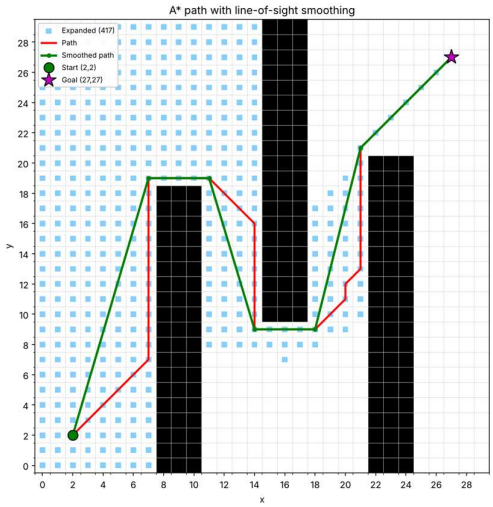


Figure 3: A path (red) and its line-of-sight smoothed version (green) over the A* expanded cells.*

Table 2: Raw and smoothed A path.*

Metric	A* (raw)	A* + smoothing
Path length	60.0416	57.0149
Number of turns	10	5
Total heading change [rad]	9.4248	5.7097
Waypoints	-	(2,2) (7,19) (11,19) (14,9) (18,9) (21,21) (27,27)

Discussion (part e)

- **Same cost, fewer nodes.** Both algorithms return the optimal cost 60.04. A* expanded 417 nodes against 610 (32% fewer) because it orders nodes by f = g + h. Dijkstra (h = 0) grows a uniform cost disc, including cells behind the start, while the Euclidean h pushes A* towards the goal. The heuristic never overestimates and is consistent, so optimality is kept.
- The saving is moderate because the walls force a detour and h ignores obstacles, so A* still explores around R1, R2 and R3 before finding the gaps.
- **Smoothness.** The raw Dijkstra path is the smoother one here (7 turns, 7.07 rad against 10 turns, 9.42 rad). Many grid paths have exactly the same cost, so this is only a tie-breaking effect; A* happens to alternate straight and diagonal steps along its f-contour. Both are limited to 45-degree headings.
- **Line-of-sight shortcutting** gives the smoothest path: length 57.01 (5.0% shorter), 5 turns and 5.71 rad, because straight segments between visible vertices replace the grid staircase. It is not guaranteed to be the true shortest path.

Task 2: Joint-space trajectory of a 2-link planar manipulator

Problem and assumptions

l1 = 1.0 m, l2 = 0.8 m, theta1 from the x-axis, theta2 relative to link 1. Move from (30 deg, 45 deg) to (60 deg, 90 deg), starting and stopping at rest, cubic polynomial per joint, T = 3 s, sampling time 0.01 s (301 samples).

For each joint q(t) = a0 + a1 t + a2 t^2 + a3 t^3 with q(0) = q0, q(T) = qf and zero end velocities, which gives:

a0 = q0, a1 = 0, a2 = 3 (qf - q0) / T^2, a3 = -2 (qf - q0) / T^3
Forward kinematics: x = l1 cos(th1) + l2 cos(th1 + th2), y = l1 sin(th1) + l2 sin(th1 + th2)

Code

```
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

L1, L2 = 1.0, 0.8                    # link lengths [m]
Q0 = np.radians([30.0, 45.0])        # start (theta1, theta2) [rad]
QF = np.radians([60.0, 90.0])        # end   (theta1, theta2) [rad]
T, DT = 3.0, 0.01                    # total time, sampling time [s]

def cubic_coeffs(q0, qf, T):
    """Return [a0, a1, a2, a3] for one joint (rest-to-rest)."""
    d = qf - q0
    return np.array([q0, 0.0, 3.0 * d / T**2, -2.0 * d / T**3])

def cubic_eval(a, t):
    """Position, velocity, acceleration of the cubic with coefficients a."""
    q = a[0] + a[1] * t + a[2] * t**2 + a[3] * t**3
    qd = a[1] + 2 * a[2] * t + 3 * a[3] * t**2
    qdd = 2 * a[2] + 6 * a[3] * t
    return q, qd, qdd

def fk(q1, q2):
    """Forward kinematics: positions of elbow and end-effector."""
    xe, ye = L1 * np.cos(q1), L1 * np.sin(q1)
    x = xe + L2 * np.cos(q1 + q2)
    y = ye + L2 * np.sin(q1 + q2)
    return xe, ye, x, y

def trajectory():
    """Cubic trajectory of both joints on the sampled time grid."""
    t = np.arange(0.0, T + DT / 2, DT)
    coeffs = [cubic_coeffs(Q0[i], QF[i], T) for i in range(2)]
    q = np.zeros((2, t.size))
    qd = np.zeros_like(q)
    qdd = np.zeros_like(q)
    for i in range(2):
        q[i], qd[i], qdd[i] = cubic_eval(coeffs[i], t)
    return t, q, qd, qdd, coeffs

def main():
    t, q, qd, qdd, coeffs = trajectory()
    deg = np.degrees

    # (a) coefficients (angles in degrees)
    print("(a) Cubic coefficients [deg, deg/s, deg/s^2, deg/s^3]")
    for i, a in enumerate(coeffs):
        print(f"  joint {i + 1}: a0={deg(a[0]):.4f}  a1={deg(a[1]):.4f}  "
              f"a2={deg(a[2]):.4f}  a3={deg(a[3]):.4f}")

    # (b) peaks
    peaks = []
    for i in range(2):
        k_v = np.argmax(np.abs(qd[i]))
        k_a = np.argmax(np.abs(qdd[i]))
        peaks.append({"joint": i + 1,
                      "vel": float(deg(qd[i][k_v])), "t_vel": float(t[k_v]),
                      "acc": float(deg(qdd[i][k_a])), "t_acc": float(t[k_a]),
                      "acc_end": float(deg(qdd[i][-1]))})
        print(f"  joint {i + 1}: peak vel {peaks[-1]['vel']:.2f} deg/s at t={t[k_v]:.2f} s, "
              f"peak |acc| {abs(peaks[-1]['acc']):.2f} deg/s^2 at t={t[k_a]:.2f} s")

    fig, axs = plt.subplots(3, 1, figsize=(8, 10), sharex=True)
    labels = [r"$\theta_1$", r"$\theta_2$"]
    cols = ["tab:blue", "tab:red"]
    for i in range(2):
        axs[0].plot(t, deg(q[i]), color=cols[i], label=labels[i])
        axs[1].plot(t, deg(qd[i]), color=cols[i], label=labels[i])
        axs[2].plot(t, deg(qdd[i]), color=cols[i], label=labels[i])
        # mark peak values
        k_v = np.argmax(np.abs(qd[i]))
        axs[1].plot(t[k_v], deg(qd[i][k_v]), "o", color=cols[i], mec="k", ms=8)
        axs[1].annotate(f"{deg(qd[i][k_v]):.1f} deg/s", (t[k_v], deg(qd[i][k_v])),
                        textcoords="offset points", xytext=(8, 6), fontsize=9)
        for kk in (0, t.size - 1):
            axs[2].plot(t[kk], deg(qdd[i][kk]), "o", color=cols[i], mec="k", ms=8)
            axs[2].annotate(f"{deg(qdd[i][kk]):.1f}", (t[kk], deg(qdd[i][kk])),
                            textcoords="offset points",
                            xytext=(8 if kk == 0 else -40, 6 if kk == 0 else 6), fontsize=9)
        axs[0].plot([0, T], deg([q[i][0], q[i][-1]]), "o", color=cols[i], mec="k", ms=6)
    axs[0].set_ylabel("Joint angle [deg]")
    axs[1].set_ylabel("Joint velocity [deg/s]")
    axs[2].set_ylabel(r"Joint acceleration [deg/s$^2$]")
    axs[2].set_xlabel("Time [s]")
    axs[0].set_title("Cubic joint trajectories (peak values marked)")
```

```
    for ax in axs:
        ax.grid(alpha=0.4)
        ax.legend(loc="best")
    fig.tight_layout()
    fig.savefig("task2_joints.png", dpi=150)
    plt.close(fig)

    # (c) forward kinematics
    xe, ye, x, y = fk(q[0], q[1])
    fig, ax = plt.subplots(figsize=(8, 4.5))
    ax.plot(t, x, label="x(t)")
    ax.plot(t, y, label="y(t)")
    ax.set_xlabel("Time [s]")
    ax.set_ylabel("End-effector coordinate [m]")
    ax.set_title("End-effector coordinates vs time")
    ax.grid(alpha=0.4)
    ax.legend()
    fig.tight_layout()
    fig.savefig("task2_ee_time.png", dpi=150)
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(8, 8))
    ax.plot(x, y, "k--", lw=2, label="End-effector path")
    ax.plot(x[0], y[0], "go", ms=11, mec="k", label=f"Start ({x[0]:.3f}, {y[0]:.3f}) m")
    ax.plot(x[-1], y[-1], "r*", ms=16, mec="k", label=f"End ({x[-1]:.3f}, {y[-1]:.3f}) m")
    snaps = [0.0, T / 4, T / 2, 3 * T / 4, T]
    cm = plt.cm.viridis(np.linspace(0, 0.9, len(snaps)))
    for c, ts in zip(cm, snaps):
        k = int(round(ts / DT))
        ax.plot([0, xe[k], x[k]], [0, ye[k], y[k]], "-o", color=c, lw=2.5, ms=6,
                label=f"Arm at t = {ts:.2f} s")
    ax.plot(0, 0, "ks", ms=9, label="Base")
    ax.set_aspect("equal")
    ax.set_xlabel("x [m]")
    ax.set_ylabel("y [m]")
    ax.set_title("Arm configurations and end-effector path in the x-y plane")
    ax.grid(alpha=0.4)
    ax.set_xlim(-0.4, 1.9)
    ax.legend(loc="lower right", fontsize=8)
    fig.tight_layout()
    fig.savefig("task2_ee_path.png", dpi=150)
    plt.close(fig)

    ee = {"start": [float(x[0]), float(y[0])], "end": [float(x[-1]), float(y[-1])],
          "x_range": [float(x.min()), float(x.max())],
          "y_range": [float(y.min()), float(y.max())]}
    print("(c) Start", ee["start"], "End", ee["end"])

    json.dump({"coeffs_deg": [[float(deg(c)) for c in a] for a in coeffs],
               "peaks": peaks, "ee": ee, "n_samples": int(t.size)},
              open("results_task2.json", "w"), indent=1)


if __name__ == "__main__":
    main()
```

Results (part a): coefficients

Table 3: Cubic coefficients (angles in degrees).

Joint	a0 [deg]	a1 [deg/s]	a2 [deg/s^2]	a3 [deg/s^3]
Joint 1	30.0000	0.0000	10.0000	-2.2222
Joint 2	45.0000	0.0000	15.0000	-3.3333

Results (part b): joint angle, velocity and acceleration

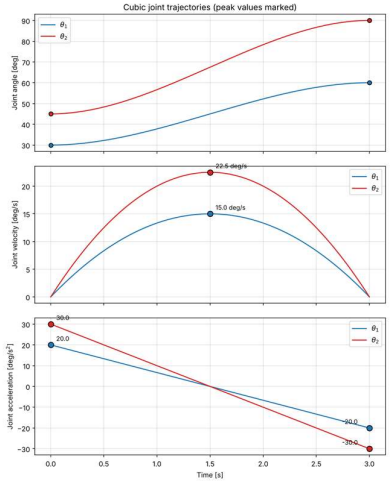


Figure 4: Joint angles, velocities and accelerations against time, with peak values marked.

Table 4: Peak values.

Joint	Peak velocity [deg/s]	Time [s]	Peak |acceleration| [deg/s^2]	Occurs at [s]
Joint 1	15.00	1.50	20.00	0 (+) and 3 (-)
Joint 2	22.50	1.50	30.00	0 (+) and 3 (-)

Results (part c): end-effector motion

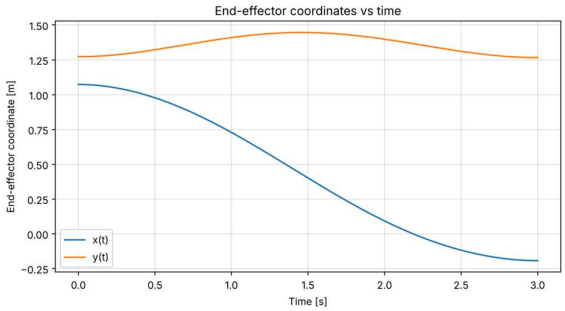


Figure 5: End-effector coordinates x(t) and y(t) from forward kinematics.

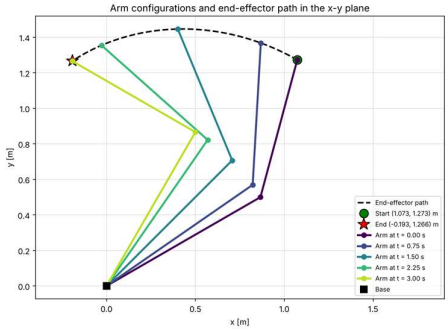


Figure 6: End-effector path in the x-y plane with start, end and arm configurations at t = 0, T/4, T/2, 3T/4 and T.

Table 5: End-effector positions.

Point	x [m]	y [m]
Start (t = 0)	1.0731	1.2727
End (t = 3 s)	-0.1928	1.2660
Range during motion	-0.193 to 1.073	1.266 to 1.447

Discussion

- Velocity is a parabola with its peak at T/2 (1.5 x total change / T: 15.00 and 22.50 deg/s). Acceleration is linear in time, with maximum magnitude 6 x change / T^2 (20.00 and 30.00 deg/s^2) at t = 0 and t = T.
- Both joints reach their targets at the same time and have the same normalised profile; joint 2 is simply 1.5 times larger because it moves 45 deg against 30 deg.
- Acceleration jumps from zero to its peak at the start and end, so jerk is infinite there. A quintic polynomial would remove this.
- A smooth joint-space motion does not give a straight line in Cartesian space: the end-effector follows a curved arc.

Task 3: Sampling-based planning (RRT)

Problem and assumptions

100 x 100 workspace, start (5, 5), goal (95, 95), circular obstacles O1 (30, 30, r 10), O2 (60, 70, r 12), O3 (70, 30, r 8), O4 (20, 75, r 9), O5 (50, 50, r 7). RRT is implemented with step size 5, goal bias 5 %, goal tolerance 5 and at most 5000 iterations. A node is accepted only if the segment from its parent is collision free.

- Assumed: uniform sampling over [0, 100] x [0, 100]; nearest node by linear search; the tree is steered at most 5 units toward the sample.
- Assumed: the search stops when a new node is within 5 units of the goal and the final connecting segment to the goal is collision free; the goal is then appended as the last node.
- Assumed: random seed 1 for the plots, so the figures can be reproduced. Because RRT is random, 30 further seeds are also reported.
- Collision check (part a): the distance from the circle centre to the closest point of the segment (clamped to the segment) must be larger than the radius.

Code

```
import json
import math
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Circle

START, GOAL = (5.0, 5.0), (95.0, 95.0)
WORLD = 100.0
OBSTACLES = [(30, 30, 10), (60, 70, 12), (70, 30, 8), (20, 75, 9), (50, 50, 7)]  # (cx, cy, r)
STEP, GOAL_BIAS, GOAL_TOL, MAX_ITER = 5.0, 0.05, 5.0, 5000
SEED = 1  # assumption: fixed seed for a reproducible figure

def segment_hits_circle(p, q, c, r):
    """True if segment p-q intersects the disc with centre c and radius r.
    Uses the distance from the centre to the closest point of the segment."""
    px, py = p
    dx, dy = q[0] - px, q[1] - py
    seg2 = dx * dx + dy * dy
    if seg2 == 0.0:
        t = 0.0
    else:
        t = ((c[0] - px) * dx + (c[1] - py) * dy) / seg2
        t = max(0.0, min(1.0, t))          # clamp to the segment
    cx, cy = px + t * dx, py + t * dy      # closest point on the segment
    return math.hypot(c[0] - cx, c[1] - cy) <= r

def segment_free(p, q, obstacles=OBSTACLES):
    """True if the segment p-q is collision free w.r.t. all circular obstacles."""
    return not any(segment_hits_circle(p, q, (ox, oy), r) for ox, oy, r in obstacles)

def point_free(p):
    return segment_free(p, p)

def rrt(rng):
    nodes = [START]
    parent = [-1]
    samples = []
    for it in range(1, MAX_ITER + 1):
        # sample with 5 % goal bias
        if rng.random() < GOAL_BIAS:
            s = GOAL
        else:
            s = (rng.uniform(0, WORLD), rng.uniform(0, WORLD))
        samples.append(s)
        # nearest node (linear search)
        d2 = [(n[0] - s[0]) ** 2 + (n[1] - s[1]) ** 2 for n in nodes]
        i_near = int(np.argmin(d2))
        near = nodes[i_near]
        dist = math.sqrt(d2[i_near])
        if dist == 0.0:
            continue
        # steer: move at most STEP towards the sample
        k = min(1.0, STEP / dist)
        new = (near[0] + k * (s[0] - near[0]), near[1] + k * (s[1] - near[1]))
        if not segment_free(near, new):
            continue
        nodes.append(new)
        parent.append(i_near)
        # goal test: within tolerance and the final connection is collision free
        if math.hypot(new[0] - GOAL[0], new[1] - GOAL[1]) <= GOAL_TOL and segment_free(new, GOAL):
            nodes.append(GOAL)
            parent.append(len(nodes) - 2)
            path, i = [], len(nodes) - 1
            while i != -1:
                path.append(nodes[i])
                i = parent[i]
            return path[::-1], nodes, parent, samples, it
    return None, nodes, parent, samples, MAX_ITER
```

```
def path_len(path):
    return sum(math.dist(path[i], path[i + 1]) for i in range(len(path) - 1))

def draw_scene(ax):
    for ox, oy, r in OBSTACLES:
        ax.add_patch(Circle((ox, oy), r, fc="dimgray", ec="k", alpha=0.85))
    ax.plot(*START, "go", ms=11, mec="k", label="Start (5, 5)", zorder=5)
    ax.plot(*GOAL, "m*", ms=17, mec="k", label="Goal (95, 95)", zorder=5)
    ax.add_patch(Circle(GOAL, GOAL_TOL, fc="none", ec="m", ls=":", lw=1))
    ax.set_xlim(0, WORLD)
    ax.set_ylim(0, WORLD)
    ax.set_aspect("equal")
    ax.set_xlabel("x [units]")
    ax.set_ylabel("y [units]")

def main():
    assert point_free(START) and point_free(GOAL)
    rng = np.random.default_rng(SEED)
    path, nodes, parent, samples, iters = rrt(rng)
    assert path is not None, "RRT failed within the iteration limit"
    length = path_len(path)
    straight = math.dist(START, GOAL)
    print(f"Seed {SEED}: iterations {iters}, nodes {len(nodes)}, path length {length:.2f}, "
          f"straight-line {straight:.2f}, ratio {length / straight:.3f}, waypoints {len(path)}")

    # (c-i) samples and tree
    fig, ax = plt.subplots(figsize=(8, 8))
    sm = np.array(samples)
    ax.scatter(sm[:, 0], sm[:, 1], s=4, c="lightgray", label=f"Random samples ({len(samples)})")
    for i in range(1, len(nodes)):
        a, b = nodes[parent[i]], nodes[i]
        ax.plot([a[0], b[0]], [a[1], b[1]], "-", color="tab:blue", lw=0.8)
    ax.plot([], [], "-", color="tab:blue", label=f"RRT tree ({len(nodes)} nodes)")
    draw_scene(ax)
    ax.set_title("RRT: random samples and expanding tree")
    ax.legend(loc="upper left", fontsize=8)
    fig.tight_layout()
    fig.savefig("task3_rrt_tree.png", dpi=150)
    plt.close(fig)

    # (c-ii) final path
    fig, ax = plt.subplots(figsize=(8, 8))
    for i in range(1, len(nodes)):
        a, b = nodes[parent[i]], nodes[i]
        ax.plot([a[0], b[0]], [a[1], b[1]], "-", color="lightsteelblue", lw=0.6)
    ax.plot([], [], "-", color="lightsteelblue", label="RRT tree")
    p = np.array(path)
    ax.plot(p[:, 0], p[:, 1], "r-o", lw=2.5, ms=4, label=f"Final path (length {length:.1f})")
    draw_scene(ax)
    ax.set_title("RRT: final collision-free path")
    ax.legend(loc="upper left", fontsize=8)
    fig.tight_layout()
    fig.savefig("task3_rrt_path.png", dpi=150)
    plt.close(fig)

    # extra: repeatability over 30 random seeds
    stats = []
    for sd in range(100, 130):
        pth, nd, _, _, it = rrt(np.random.default_rng(sd))
        if pth:
            stats.append((it, len(nd), path_len(pth)))
    st = np.array(stats)
    print(f"30 seeds: success {len(st)}/30, iterations mean {st[:, 0].mean():.0f}, "
          f"nodes mean {st[:, 1].mean():.0f}, length mean {st[:, 2].mean():.1f} "
          f"(min {st[:, 2].min():.1f}, max {st[:, 2].max():.1f})")

    json.dump({"iters": iters, "nodes": len(nodes), "length": length, "straight": straight,
               "waypoints": len(path), "path": [list(map(float, q)) for q in path],
               "n_seeds": 30, "success": int(len(st)),
               "it_mean": float(st[:, 0].mean()), "nodes_mean": float(st[:, 1].mean()),
               "len_mean": float(st[:, 2].mean()), "len_min": float(st[:, 2].min()),
               "len_max": float(st[:, 2].max()), "seed": SEED},
              open("results_task3.json", "w"), indent=1)

if __name__ == "__main__":
    main()
```

Results (part c)

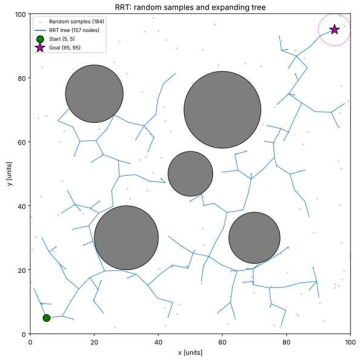


Figure 7: RRT: random samples (grey dots) and the tree (blue).

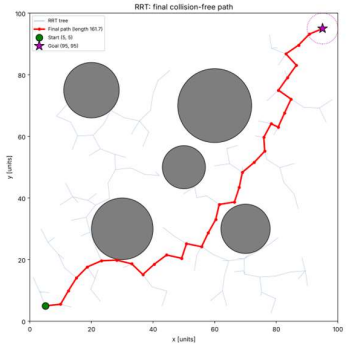


Figure 8: RRT: final collision-free path (red) over the tree, obstacles, start and goal (dotted circle = goal tolerance).

Results (part c)

Table 6: RRT result for the plotted run (seed 1).

Quantity	Value
Iterations until goal reached	184
Tree nodes	157
Waypoints on the path	34
Path length [units]	161.70
Straight-line distance start-goal [units]	127.28
Path length / straight-line distance	1.270

Table 7: Repeatability over 30 different seeds.

Quantity	Value
Success rate	30 / 30
Mean iterations	168
Mean number of nodes	135
Mean path length [units]	164.02
Shortest / longest path [units]	148.28 / 187.73

Discussion

- RRT found a collision-free path in 184 iterations (limit 5000) and succeeded in all 30 extra runs, so it is fast and reliable in this fairly open workspace.
- The path is 161.70 units, 27% longer than the straight line (127.28). It is jagged and varies from run to run (148.28 to 187.73): RRT finds a feasible path, not an optimal one, and the tree is biased toward unexplored space rather than toward short paths.
- The 5 % goal bias helps the tree reach the goal region without slowing exploration. The path could be shortened by post-processing (shortcutting) or by using RRT*.

Task 4: Dynamics of a 2-link planar manipulator

Problem and assumptions

Same arm as Task 2 in the vertical plane: l1 = 1.0 m, l2 = 0.8 m, m1 = 2.0 kg, m2 = 1.5 kg, uniform slender rods with the centre of mass at mid-link (lc = l/2) and I = m l^2 / 12, g = 9.81 m/s^2 along -y, no friction. The task sheet lists parts (a) and (c), so these two are reported.

Equations of motion (Lagrange), q = [theta1, theta2]:

```
M(q) qdd + c(q, qd) + G(q) = tau
M11 = m1 lc1^2 + I1 + m2 (l1^2 + lc2^2 + 2 l1 lc2 cos q2) + I2
M12 = M21 = m2 (lc2^2 + l1 lc2 cos q2) + I2,     M22 = m2 lc2^2 + I2
h = m2 l1 lc2 sin q2,   c = [ -h (2 qd1 qd2 + qd2^2),  h qd1^2 ]
G1 = (m1 lc1 + m2 l1) g cos q1 + m2 lc2 g cos(q1 + q2),   G2 = m2 lc2 g cos(q1 + q2)
Energy: T = 0.5 qd' M qd,  V = m1 g lc1 sin q1 + m2 g (l1 sin q1 + lc2 sin(q1 + q2))
```

- Assumed: potential energy reference y = 0 at the shoulder, so the initial total energy is 0 J.
- Assumed: solve_ivp with RK45, rtol = atol = 1e-10, output every 0.01 s. Peak torque means the largest absolute torque.
- Forward dynamics (a): tau = 0, start at rest at q = (0, 0) (arm horizontal, fully stretched), 5 s. Inverse dynamics (c): the Task 2 cubic trajectory (T = 3 s) with analytic velocity and acceleration.

Code

```
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp          # RK45 = Python equivalent of ode45

from task2_joint_trajectory import Q0, QF, T, cubic_coeffs, cubic_eval

L1, L2 = 1.0, 0.8
M1, M2 = 2.0, 1.5
G = 9.81
LC1, LC2 = L1 / 2, L2 / 2
I1, I2 = M1 * L1**2 / 12, M2 * L2**2 / 12

def mass_matrix(q):
    c2 = np.cos(q[1])
    m11 = M1 * LC1**2 + I1 + M2 * (L1**2 + LC2**2 + 2 * L1 * LC2 * c2) + I2
    m12 = M2 * (LC2**2 + L1 * LC2 * c2) + I2
    m22 = M2 * LC2**2 + I2
    return np.array([[m11, m12], [m12, m22]])

def coriolis(q, qd):
    h = M2 * L1 * LC2 * np.sin(q[1])
    return np.array([-h * (2 * qd[0] * qd[1] + qd[1] ** 2), h * qd[0] ** 2])

def gravity(q):
    g1 = (M1 * LC1 + M2 * L1) * G * np.cos(q[0]) + M2 * LC2 * G * np.cos(q[0] + q[1])
    g2 = M2 * LC2 * G * np.cos(q[0] + q[1])
    return np.array([g1, g2])

def inverse_dynamics(q, qd, qdd):
    return mass_matrix(q) @ qdd + coriolis(q, qd) + gravity(q)

def forward_dynamics(q, qd, tau):
    return np.linalg.solve(mass_matrix(q), tau - coriolis(q, qd) - gravity(q))

def energy(q, qd):
    kin = 0.5 * qd @ mass_matrix(q) @ qd
    # potential energy, reference y = 0 at the shoulder
    pot = M1 * G * LC1 * np.sin(q[0]) + M2 * G * (L1 * np.sin(q[0]) + LC2 * np.sin(q[0] + q[1]))
    return kin, pot

def fk(q):
    xe, ye = L1 * np.cos(q[0]), L1 * np.sin(q[0])
    return xe, ye, xe + L2 * np.cos(q[0] + q[1]), ye + L2 * np.sin(q[0] + q[1])

def main():
    out = {}

        def rhs(t, s):
        q, qd = s[:2], s[2:]
        return np.concatenate([qd, forward_dynamics(q, qd, np.zeros(2))])

    t_eval = np.arange(0.0, 5.0 + 1e-9, 0.01)
    sol = solve_ivp(rhs, (0, 5), [0, 0, 0, 0], method="RK45", t_eval=t_eval,
                    rtol=1e-10, atol=1e-10)
    t, q, qd = sol.t, sol.y[:2], sol.y[2:]
    E = np.array([sum(energy(q[:, k], qd[:, k])) for k in range(t.size)])
    KE = np.array([energy(q[:, k], qd[:, k])[0] for k in range(t.size)])
    PE = np.array([energy(q[:, k], qd[:, k])[1] for k in range(t.size)])
    drift = float(np.max(np.abs(E - E[0])))
    print(f"(a) E0 = {E[0]:.4f} J, max |E-E0| = {drift:.2e} J, "
          f"theta1 range [{np.degrees(q[0]).min():.1f}, {np.degrees(q[0]).max():.1f}] deg, "
          f"theta2 range [{np.degrees(q[1]).min():.1f}, {np.degrees(q[1]).max():.1f}] deg")
    out["free"] = {"E0": float(E[0]), "drift": drift,
                   "th1": [float(np.degrees(q[0]).min()), float(np.degrees(q[0]).max())],
```

```
                "th2": [float(np.degrees(q[1]).min()), float(np.degrees(q[1]).max())],
                "KE_max": float(KE.max()), "PE_min": float(PE.min())}

    xe, ye, x, y = np.array([fk(q[:, k]) for k in range(t.size)]).T
    fig = plt.figure(figsize=(11, 9))
    gs = fig.add_gridspec(2, 2, height_ratios=[1, 1.25])
    axs = [fig.add_subplot(gs[0, 0]), fig.add_subplot(gs[0, 1]), fig.add_subplot(gs[1, :])]
    axs[0].plot(t, np.degrees(q[0]), label=r"$\theta_1$")
    axs[0].plot(t, np.degrees(q[1]), label=r"$\theta_2$")
    axs[0].set_xlabel("Time [s]")
    axs[0].set_ylabel("Joint angle [deg]")
    axs[0].set_title("Free motion (tau = 0): joint angles")
    axs[0].grid(alpha=0.4)
    axs[0].legend()
    axs[1].plot(t, x, label="x")
    axs[1].plot(t, y, label="y")
    axs[1].set_xlabel("Time [s]")
    axs[1].set_ylabel("End-effector coordinate [m]")
    axs[1].set_title("End-effector coordinates")
    axs[1].grid(alpha=0.4)
    axs[1].legend()
    axs[2].plot(x, y, "b-", lw=1.2, label="End-effector path")
    axs[2].plot(x[0], y[0], "go", mec="k", ms=9, label="Start (1.8, 0) m")
    axs[2].plot(x[-1], y[-1], "r*", mec="k", ms=13, label="End (t = 5 s)")
    axs[2].plot(0, 0, "ks", label="Base")
    axs[2].set_aspect("equal")
    axs[2].set_xlabel("x [m]")
    axs[2].set_ylabel("y [m]")
    axs[2].set_title("End-effector path (x-y plane)")
    axs[2].grid(alpha=0.4)
    axs[2].legend(fontsize=8, loc="upper right")
    fig.tight_layout()
    fig.savefig("task4_free_motion.png", dpi=150)
    plt.close(fig)

    fig, axs = plt.subplots(2, 1, figsize=(8, 7), sharex=True)
    axs[0].plot(t, KE, label="Kinetic")
    axs[0].plot(t, PE, label="Potential")
    axs[0].plot(t, E, "k--", label="Total")
    axs[0].set_ylabel("Energy [J]")
    axs[0].set_title("Energy during free motion")
    axs[0].grid(alpha=0.4)
    axs[0].legend()
    axs[1].plot(t, E - E[0], "k")
    axs[1].set_xlabel("Time [s]")
    axs[1].set_ylabel("E(t) - E(0) [J]")
    axs[1].set_title("Total-energy error (conservation check)")
    axs[1].grid(alpha=0.4)
    axs[1].legend(["Total energy error"])
    fig.tight_layout()
    fig.savefig("task4_energy.png", dpi=150)
    plt.close(fig)

        tt = np.arange(0.0, T + 0.005, 0.01)
    coef = [cubic_coeffs(Q0[i], QF[i], T) for i in range(2)]
    qc = np.zeros((2, tt.size)); qdc = np.zeros_like(qc); qddc = np.zeros_like(qc)
    for i in range(2):
        qc[i], qdc[i], qddc[i] = cubic_eval(coef[i], tt)
    tau = np.array([inverse_dynamics(qc[:, k], qdc[:, k], qddc[:, k]) for k in range(tt.size)]).T
    peaks = []
    for i in range(2):
        k = int(np.argmax(np.abs(tau[i])))
        peaks.append({"joint": i + 1, "peak": float(tau[i][k]), "t": float(tt[k]),
                      "t0": float(tau[i][0]), "tT": float(tau[i][-1])})
        print(f"(c) tau{i + 1}: peak {tau[i][k]:.2f} N m at t = {tt[k]:.2f} s "
              f"(tau(0) = {tau[i][0]:.2f}, tau(T) = {tau[i][-1]:.2f})")
    out["tau_peaks"] = peaks

    fig, ax = plt.subplots(figsize=(8, 5))
    for i, c in enumerate(["tab:blue", "tab:red"]):
        ax.plot(tt, tau[i], color=c, label=rf"$\tau_{i + 1}$")
        k = int(np.argmax(np.abs(tau[i])))
        ax.plot(tt[k], tau[i][k], "o", color=c, mec="k", ms=8)
        ax.annotate(f"{tau[i][k]:.1f} N m", (tt[k], tau[i][k]), textcoords="offset points",
                    xytext=(8, 6), fontsize=9)
    ax.set_xlabel("Time [s]")
    ax.set_ylabel("Joint torque [N m]")
    ax.set_title("Inverse dynamics: torques for the Task 2 cubic trajectory (peaks marked)")
    ax.grid(alpha=0.4)
    ax.legend()
    fig.tight_layout()
    fig.savefig("task4_torques.png", dpi=150)
    plt.close(fig)

    # verification: apply tau(t) in forward dynamics and compare with the reference
    def rhs_tau(t, s):
        qr = np.array([cubic_eval(coef[i], t)[0] for i in range(2)])
        qdr = np.array([cubic_eval(coef[i], t)[1] for i in range(2)])
        qddr = np.array([cubic_eval(coef[i], t)[2] for i in range(2)])
        return np.concatenate([s[2:], forward_dynamics(s[:2], s[2:], inverse_dynamics(qr, qdr, qddr))])

    v = solve_ivp(rhs_tau, (0, T), [*Q0, 0, 0], method="RK45", t_eval=tt, rtol=1e-10, atol=1e-10)
    err = float(np.degrees(np.max(np.abs(v.y[:2] - qc))))
    print(f"(c) check: max joint error forward(tau) vs reference = {err:.2e} deg")
    out["check_err_deg"] = err

    json.dump(out, open("results_task4.json", "w"), indent=1)


if __name__ == "__main__":
    main()
```

Results (part a): free motion and energy

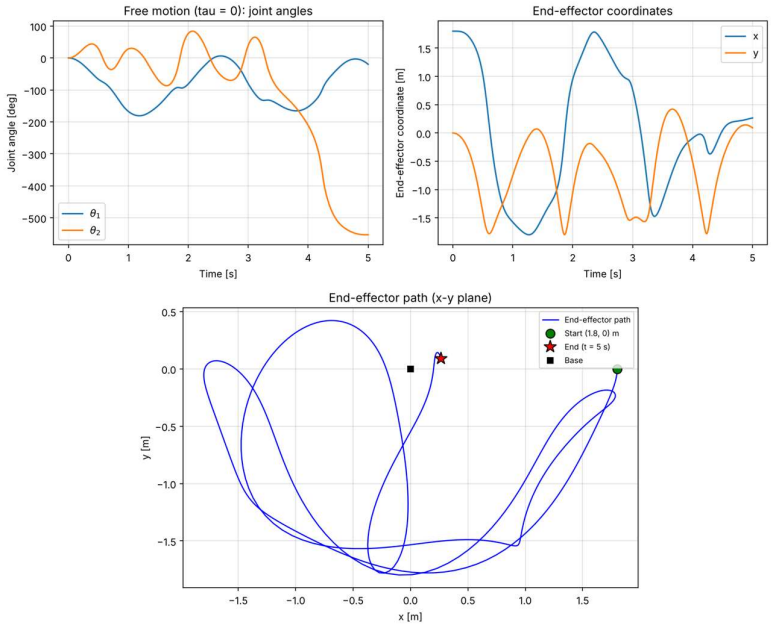


Figure 9: Free motion with tau = 0: joint angles, end-effector coordinates and end-effector path.

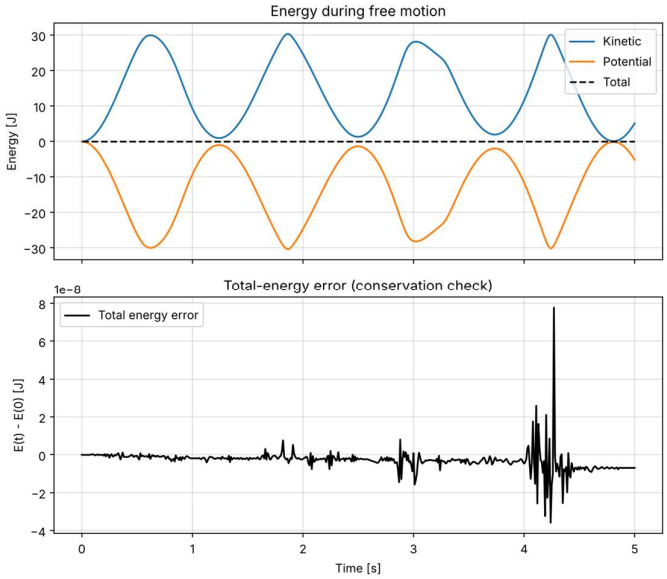


Figure 10: Kinetic, potential and total energy during free motion, and the total-energy error.

Table 8: Energy conservation check.

Quantity	Value
Initial total energy E(0)	0.00 J
Maximum |E(t) - E(0)| over 5 s	7.77e-8 J
Maximum kinetic energy	30.37 J
Minimum potential energy	-30.37 J
Range of theta1	-180.7 to 6.4 deg
Range of theta2	-552.8 to 83.8 deg

Results (part c): joint torques for the cubic trajectory

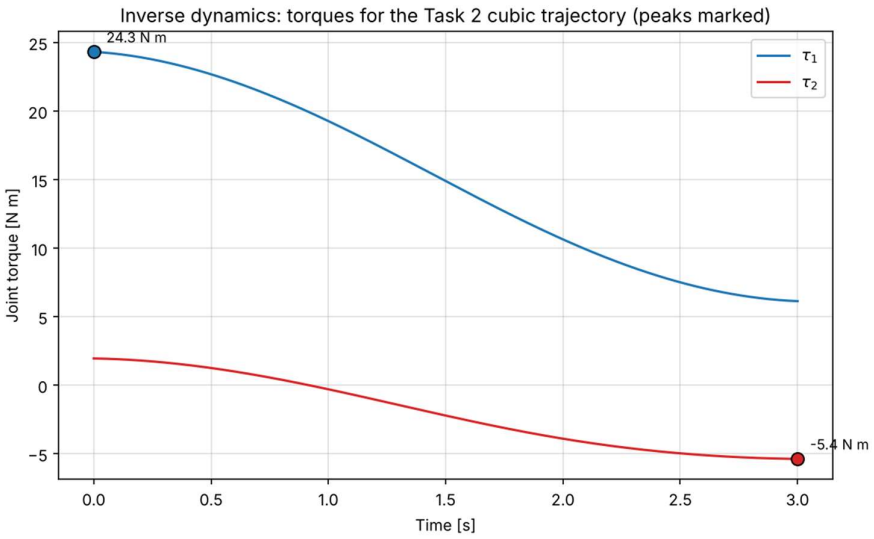


Figure 11: Joint torques required to follow the Task 2 cubic trajectory (peak values marked).

Table 9: Torque results.

Joint	tau(0) [N m]	tau(T) [N m]	Peak |tau| [N m]	Peak at t [s]
Joint 1	24.32	6.13	24.32	0.00
Joint 2	1.95	-5.38	-5.38	3.00

Check: applying the computed torques in the forward dynamics reproduces the reference joint trajectory with a maximum error of 6.1e-5 deg, which confirms that the inverse and forward models are consistent.

Discussion

- With no torque and no friction the total energy stays constant: the maximum error is 7.8e-8 J, which is integration error only. The arm falls from the horizontal position and exchanges potential and kinetic energy; the motion is irregular and non-periodic (a double pendulum is chaotic), and joint 2 even swings through about 553 deg.
- The torques of the slow Task 2 motion are mostly gravity torques, because the accelerations are small (at most 0.5 rad/s^2). Joint 1 needs the larger torque, with a peak of 24.32 N m at t = 0 (the arm is closest to horizontal and carries the whole arm). Joint 2 peaks at 5.38 N m in magnitude at t = T and changes sign.
- The torque curves are smooth, but the torque is not zero at the start and end of the motion: holding the arm still against gravity needs a non-zero torque.